

Collective Communication Algorithms for Distributed Training

Jo Jojo, Jeffrey Liang

May 11, 2026

1 Introduction

When training a large neural network across multiple GPUs, each device computes gradients on its own slice of data. Before the next training step, all devices must agree on the same aggregated gradient values which requires an element-wise sum across every worker. This common method is called **AllReduce**, and its performance is often the dominant cost in large-scale training.

The simplest approach routes all gradients through a single *parameter server*: workers upload their arrays, the server sums them, and sends the result back. This is easy to implement but has an obvious flaw: the server’s bandwidth grows linearly with the number of workers, making it a hard bottleneck.

As such, we wanted to address this issue and implement a paper that inspired us to pursue a new popularized method called Ring-All Reduce. Our project implements and benchmarks these two AllReduce strategies in Go:

1. **Naive (Parameter Server) AllReduce** — a central reducer that receives from all workers and broadcasts back.
2. **Ring-AllReduce** — a peer-to-peer algorithm, popularized by Baidu Research [1] and used in NVIDIA’s NCCL, in which every node only talks to its two immediate neighbors and total communication cost per node is nearly constant regardless of worker count.

We simulate both algorithms with real TCP communication on a single machine, using a per-link token-bucket rate limiter to reproduce the dedicated-bandwidth assumption that underpins Ring-AllReduce’s theoretical guarantees. Our results confirm the theoretical predictions: Ring-AllReduce’s latency grows slowly with N , while the naive approach degrades linearly and ultimately crashes at large N values due to OS socket buffer exhaustion.

2 Design and Implementation

2.1 System Overview

The project is written entirely in Go using the standard library (`net`, `sync`, `encoding/binary`). We didn’t use any external dependencies.

2.2 Rate Limiter

A per-link token-bucket rate limiter (`RateLimiter`) is implemented in the `proto` package and wrapped around outbound connections via `ThrottledConn`. The goal is to simulate each GPU-to-GPU link having its own bandwidth, instead of every connection unrealistically sharing loopback at memory speed. The sender is responsible for respecting link capacity; the receiver reads as fast as possible, matching what real NIC behavior is like. Setting `-bw 100` limits each link to 100 MB/s. Without this, Ring-AllReduce’s advantage is obscured because the parallel chunk transfers do not behave like separate links.

2.3 Naive AllReduce

Reducer (`reducer/main.go`):

1. Listens on a fixed TCP port and accepts connections from all N workers.
2. Receives each worker’s gradient array concurrently using one goroutine per connection, protected by a mutex.
3. Sums all arrays element-wise into a result buffer.
4. Broadcasts the result back to each worker sequentially over rate-limited connections.

Worker (`worker/main.go`): Connects to the reducer, sends its local gradient array, and reads back the summed result.

Bottleneck: The reducer must receive $N \times \text{array_size}$ bytes and re-send $N \times \text{array_size}$ bytes. Both phases grow linearly with N . At large N the OS socket buffer pool (`mbufs` on macOS) can be exhausted entirely. Also just wanted to say that we are simulating how GPU connections work on one computer so the crashing limit may not be realistic however scaling problems are alike.

2.4 Ring-AllReduce

Each node (`ring/main.go`) is simultaneously a client and a server. The ring topology is established closely to how the paper presents:

- Node i listens on `basePort + i` (left neighbor’s connection).
- Node i dials `basePort + (i+1) % N` (right neighbor), retrying until available.
- All nodes start listening before dialing, so startup races are handled gracefully.

The algorithm then runs in two phases:

Scatter-Reduce ($N - 1$ rounds): The array is split into N equal chunks. In each round, every node simultaneously sends one chunk to its right neighbor and receives one chunk from its left neighbor, accumulating (adding) into its local copy. After $N - 1$ rounds, every node holds the fully reduced version of exactly one chunk.

AllGather ($N - 1$ rounds): Each node passes its finished chunk around the ring. Receivers copy rather than accumulate. After $N - 1$ rounds, every node has the complete reduced array.

Why the cost is constant: Each node sends and receives exactly one chunk per round for $2(N - 1)$ rounds. Chunk size is $\text{array_size}/N$, so total bytes sent per node is:

$$2 \cdot \frac{N - 1}{N} \cdot \text{array_size} \approx 2 \cdot \text{array_size} \quad \text{as } N \rightarrow \infty$$

We can observe this constraint is completely independent of N , which is the key advantage over the naive approach.

3 Discussion and Results

3.1 Benchmark Setup

We measured completion time using Go’s `time` package around each allreduce run. All benchmarks use 1,000,000 `float64` values (≈ 8 MB per worker) with 100 MB/s of simulated per-link bandwidth. Times are reported as the last worker to finish (worst-case end-to-end from the worker’s perspective).

The benchmarks are run on a single machine using loopback TCP; the rate limiter makes each connection behave as if it has dedicated bandwidth, restoring the hardware assumption the ring algorithm relies on.

3.2 Results

N Workers	Naive (ms)	Ring (ms)
2	80	14
4	250	43
8	154	43
16	crashed	69

Table 1: AllReduce completion time (worst-case worker). 1M float64 values, 100 MB/s simulated link bandwidth.

Naive scaling: The naive approach degrades faster than linear. At $N = 2$, workers wait about 80 ms total; at $N = 4$ they wait roughly 250 ms. This super-linear growth occurs because all workers are feeding the same reducer socket simultaneously, and the broadcast phase is sequential — each additional worker adds both to the receive bottleneck and to the broadcast queue.

Ring scaling: Ring-AllReduce grows slowly. From $N = 2$ (14 ms) to $N = 4$ (43 ms) there is an increase, but the growth is driven by the number of rounds ($2(N - 1)$) rather than bandwidth, since each round only moves $\text{array_size}/N$ bytes. In a real cluster with sub-millisecond link latency, the ring line would be nearly flat.

The $N = 16$ crash: With 16 workers each writing 8 MB simultaneously into one socket, the OS socket buffer pool is exhausted and writes return `ENOBUFS`. This isn’t a bug but more of a hard scalability ceiling of the parameter-server design because our one machine setup shares OS resources.

3.3 Challenges

Deadlock in the ring: The most significant implementation challenge was preventing deadlock. If every node sends before receiving, the ring stalls because the TCP send buffer fills up and no one is draining the other end. The fix was to launch the receive concurrently in a goroutine so that each round’s send and receive overlap.

Simulating bandwidth fairly: On loopback, all goroutines share the same physical interface. Without rate limiting, the ring and the naive reducer both run at memory speed and the bandwidth advantage of the ring is not as clear. The token-bucket rate limiter on each outbound connection helps us make this simulation meaningful by showing actual differences.

4 Use of AI Tools

Where AI did well:

- Boilerplate and scaffolding: Claude generated solid initial versions of the wire format (`proto.go`), the reducer’s concurrent receive loop, and the shell scripts. These were mostly correct and saved time.

- Documentation: It helped for AI to leave general comments on certain parts of our code so we could remember what a specific function did or an unsolved issue that was momentarily skipped.

Where AI didn't work well:

- The deadlock fix: Claude initially suggested a sequential send-then-receive in each ring round. We recognized this would probably deadlock under backpressure and rewrote `sendRecv` to launch the receive concurrently. This was the correctness solution that required understanding some TCP semantics.
- Rate limiting scope: An early version we had rate-limited both sends and receives, which double-penalized bandwidth and produced incorrect timing. We debugged this by reading the token-bucket logic and restricting throttling to sends only.

5 Conclusions and Future Work

5.1 What We Learned

This project made concrete several ideas that are easy to accept abstractly but harder to internalize without implementation:

- *Bandwidth vs. latency.* Ring-AllReduce's constant-bandwidth guarantee is real and measurable even on loopback, once you impose per-link rate limits. Without the rate limiter, the advantage is invisible because all goroutines share the same physical bus.
- *The OS has limitations.* The $N = 16$ crash on the naive reducer is not a programming error — it is a genuine resource exhaustion that would occur on any machine with bounded socket buffers. The ring sidesteps this structurally, not by tuning kernel parameters.
- *Concurrency needs to be taken care of carefully.* A sequential send-then-receive in a ring deadlocks. Identifying this required reasoning about backpressure across multiple goroutines, which is the kind of reasoning that automated tools do not yet handle reliably.

5.2 Future Work

Given more time, we would perhaps try chunked pipelining, multi-machine deployment, and comparing against other collective algorithms. Chunked pipelining would split each round into smaller transfers so networking and computation could overlap more like production systems such as NCCL or Gloo. Running on actual networked hosts would also help validate whether our simulated bandwidth model matches real hardware behavior. Finally, implementing something like Tree-AllReduce would give us another comparison point for understanding the tradeoff between bandwidth cost and latency.

References

- [1] Andrew Gibiansky, “Bringing HPC Techniques to Deep Learning,” Baidu Research blog, 2017. <http://andrew.gibiansky.com/blog/machine-learning/baidu-allreduce/>